

Big Data Processing II

IA Generativa aplicada al deporte

Retrieval-Augmented Generation (RAG)

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Marzo 2026



Universidad
Rey Juan Carlos

Mapa del bloque



Sesión 1

Control del comportamiento

Prompt Engineering + Post-training



Sesión 2

Control de la información

Retrieval-Augmented Generation (RAG)



Sesión 3

Control del flujo y la acción

Agentes, Tools y Arquitecturas Cognitivas

Introducción



Un LLM modela una distribución condicional:

$$P(y \mid x)$$

donde:

- x = prompt + contexto disponible
- y = secuencia generada

Pero:

- No tiene acceso directo a bases de datos externas.
- No consulta sistemas internos.
- No distingue entre dato real y patrón aprendido.

Conclusión: conocimiento aprendido $\neq$ acceso a información actualizada.

El Prompt Engineering:

- Controla estructura.
- Reduce ambigüedad.
- Define formato.

Pero:

Prompting $\nrightarrow$ Acceso a datos externos

No modifica la base informacional del modelo.





Un sistema real dispone de:

- Tracking físico por partido.
- Métricas avanzadas (xG, PPDA, presiones).
- Informes médicos.
- Observaciones de scouting.

Pregunta crítica:

¿Ha disminuido la intensidad de presión tras acumular más de 300 minutos en 10 días?

Un LLM aislado no puede responder correctamente.

Definición de RAG



Retrieval-Augmented Generation introduce un mecanismo externo:

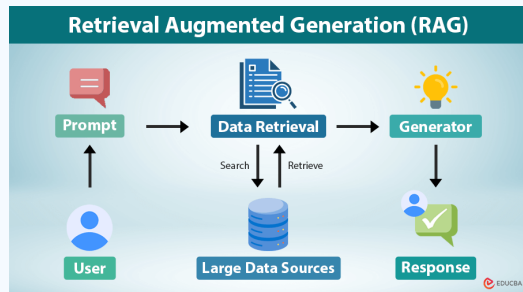
$$Output = f(Prompt + Retrieve(Query))$$

- Retrieve(Query) devuelve fragmentos relevantes.
- El LLM genera condicionado por ese contexto.

No cambia parámetros del modelo. Cambia la evidencia disponible en inferencia.

Componente	Rol
Embeddings	Representación semántica
Vector DB	Indexación eficiente
Retriever	Selección de contexto
LLM	Síntesis condicionada

RAG es una arquitectura distribuida, no una simple extensión del modelo.



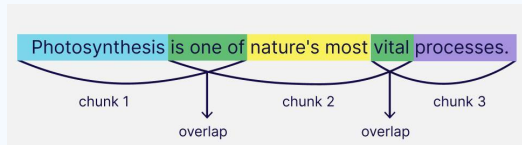
Diseño técnico de RAG

Dividir documentos implica:

- Elegir **tamaño de fragmento**.
- Definir **solapamiento**.
- Preservar **coherencia semántica**.

Errores comunes:

- Chunk demasiado grande → baja precisión.
- Chunk demasiado pequeño → pérdida de contexto.
- Solapamiento excesivo → redundancia y coste.





Un embedding es una función:

$$E : \text{Texto} \rightarrow \mathbb{R}^d$$

Permite medir similitud semántica.

Limitaciones:

- No captura bien relaciones numéricas exactas.
- Puede perder contexto temporal.
- Similaridad $\neq$ relevancia estadística.

Similitud típica:

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

Pero:

- Similaridad semántica no garantiza pertinencia.
- Sensible a redacción del query.

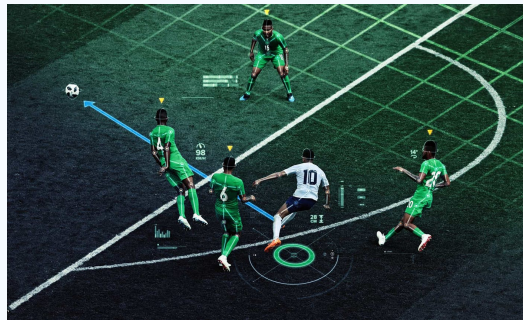
La calidad del retrieval es crítica.



Un sistema robusto debe permitir:

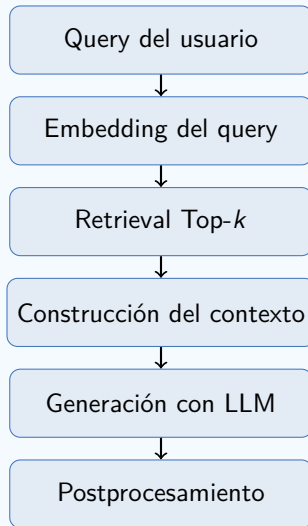
- Filtrado por temporada, jugador, competición.
- Indexación por tipo de documento.
- Actualización incremental.

Sin metadatos, el retrieval pierde precisión contextual.





Cada etapa introduce posible error.



Veracidad $\uparrow \Rightarrow$ *Coste* $\uparrow$

- Más contexto $\rightarrow$ más tokens.
- Más tokens $\rightarrow$ mayor latencia.
- Mayor latencia $\rightarrow$ menor escalabilidad.

Diseñar **RAG** es **optimizar** este equilibrio.



Aplicaciones en deporte

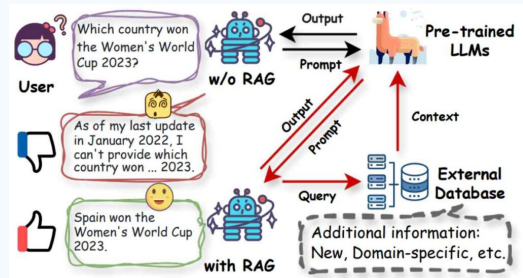
Base interna:

- Métricas por partido.
- Indicadores avanzados.
- Carga física acumulada.

Consulta:

Compara evolución de presión alta con descanso < 72h.

Requiere filtrado + recuperación + síntesis.



Documentos:

- Informes técnicos.
- Observaciones cualitativas.

Permite:

- Búsqueda semántica.
- Detección de patrones recurrentes.
- Síntesis multi-documento.





Problema:

Embeddings no gestionan bien números exactos.

Solución:

- **Recuperación textual** (búsqueda semántica en RAG).
- **Cálculo determinista externo** (fórmulas programadas).
- **Inyección de resultados estructurados** (creación de tablas numéricas).

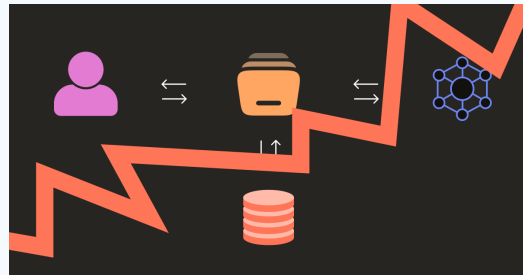
Arquitectura híbrida = mayor precisión.

Limitaciones

RAG introduce dos fuentes de error:

$$Error_{total} = Error_{retrieval} + Error_{generation}$$

Un mal retrieval no se compensa con mejor modelo.





Si el documento contiene error:

- **El modelo lo amplifica.**
- **No detecta contradicciones internas.**
- **No valida consistencia factual.**

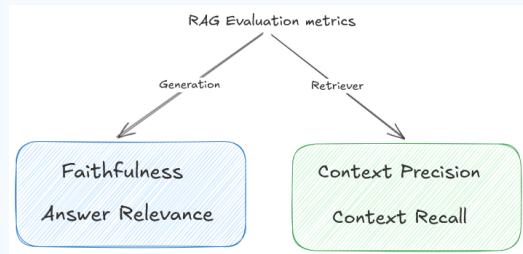
RAG mejora acceso, no garantiza verdad.

Evaluación



- Precision@k
- Recall@k
- Mean Reciprocal Rank (MRR)
- nDCG

Sin evaluar retrieval, no se puede optimizar el sistema.





Se debe medir:

- **Coherencia final.**
- **Consistencia con fuente.**
- **Estabilidad entre ejecuciones.**
- **Trazabilidad** (citar fragmentos).

RAG sin trazabilidad es **opaco** y sin estabilidad **no es confiable**.

Conclusiones



En datos dinámicos, RAG suele ser preferible.

Fine-tuning	RAG
Cambia parámetros	No cambia parámetros
Integra conocimiento	Consulta conocimiento
Coste alto inicial	Coste por consulta
Riesgo de olvido	Modelo intacto



Es una arquitectura de **acceso a información**, no un mecanismo de aprendizaje.

- RAG no hace al modelo más inteligente.
- El sistema se especializa en un contexto.
- Incrementa el tamaño del prompt.
- Afecta al número de tokens utilizados.

Siguiente paso: arquitecturas agénticas.